

main.py

Visualize

Run

1

def min_patches(coins, target):

2

3

4

miss = 1

5

i = 0

6

count = 0

7

8

while miss <= target:

9

if i < len(coins) and coins[i] <= miss:

10

miss += coins[i]

11

i += 1

12

else:

13

Add coin with value miss

14

miss += miss

15

count += 1

16

17

return count

18

19

20

Sample Input

21

coins = [1, 4, 10]

22

target = 19

23

24

print("Minimum coins to add:", min_patches(coins, target))

Output

Minimum coins to add: 2

=== Code Execution Successful ===

main.py

Visualize

Run

32

def minimum_time(jobs, k):

33

jobs.sort(reverse=True)

34

35

low = max(jobs)

36

high = sum(jobs)

37

38

while low < high:

39

mid = (low + high) // 2

40

41

if can_assign(jobs, k, mid):

42

high = mid

43

else:

44

low = mid + 1

45

46

return low

47

48

49

Sample Input

50

jobs = [3, 2, 3]

51

k = 3

52

53

print("Minimum maximum working time:", minimum_time(jobs, k))

Output

Minimum maximum working time: 3

=== Code Execution Successful ===

main.py

Visualize

Run

```
1 def kruskal(n, edges):
32     for u, v, w in edges:
33         if union(u, v):
37             if len(mst) == n - 1:
38                 break
39
40     return mst, total_weight
41
42
43 # Sample Input
44 n = 4
45 edges = [
46     (0, 1, 10),
47     (0, 2, 6),
48     (0, 3, 5),
49     (1, 3, 15),
50     (2, 3, 4)
51 ]
52
53 mst, weight = kruskal(n, edges)
54
55 print("Edges in MST:", mst)
56 print("Total weight of MST:", weight)
```

Output

Edges in MST: [(2, 3, 4), (0, 3, 5), (0, 1, 10)]
Total weight of MST: 19

=== Code Execution Successful ===

main.py

Visualize

Run

1 def dijkstra_matrix(graph, source):

9 for _ in range(n):

20 for v in range(n):

21 if graph[u][v] != float('inf') and not visited[v]:

24 if not u in dist:

26

27 return dist

28

29

30 # Sample Input

31 INF = float('inf')

32

33 graph = [

34 [0, 10, 3, INF, INF],

35 [INF, 0, 1, 2, INF],

36 [INF, 4, 0, 8, 2],

37 [INF, INF, INF, 0, 7],

38 [INF, INF, INF, 9, 0]

39]

40

41 source = 0

42

43 print("Shortest distances:", dijkstra_matrix(graph, source))

Output

Shortest distances: [0, 7, 3, 9, 5]

=== Code Execution Successful ===

main.py

Visualize

Run

3

def dijkstra_edge_list(n, edges, source, target):

...

30

return dist[target]

31

32

33 # Sample Input

34 n = 6

35 edges = [

36 (0, 1, 7),

37 (0, 2, 9),

38 (0, 5, 14),

39 (1, 2, 10),

40 (1, 3, 15),

41 (2, 3, 11),

42 (2, 5, 2),

43 (3, 4, 6),

44 (4, 5, 9)

45]

46

47 source = 0

48 target = 4

49

50 print("Shortest distance:", dijkstra_edge_list(n, edges, source, target))

Output

Shortest distance: 26

=== Code Execution Successful ===

main.py

Visualize

Run

3

def huffman_codes(characters, frequencies):

21

def generate(node, code):

24

if left is None and right is None:

25

codes[chars] = code

26

return

27

generate(left, code + "0")

28

generate(right, code + "1")

29

30

31

generate(root, "")

32

33

return codes

34

35

36

Sample Input

37

characters = ['a', 'b', 'c', 'd']

38

frequencies = [5, 9, 12, 13]

39

40

codes = huffman_codes(characters, frequencies)

41

42

for char in characters:

43

print(char, ":", codes[char])

Output

a : 00

b : 01

c : 10

d : 11

=== Code Execution Successful ===

main.py

Visualize

Run

1

def huffman_decode(codes, encoded_string):

7

for bit in encoded_string:

8

current += bit

9

if current in reverse_codes:

10

result += reverse_codes[current]

11

current = ""

12

return result

13

14

15

16

17 # Huffman codes from Test Case 1

18 codes = {

19 'a': '110',

20 'b': '10',

21 'c': '0',

22 'd': '111'

23 }

24

25 encoded_string = "1101100111110"

26

27 print("Decoded message:", huffman_decode(codes, encoded_string))

Output

Decoded message: aacda

=== Code Execution Successful ===



main.py

Visualize

Run



JS

TS

```
1 def maximum_weight(weights, capacity):
2     weights.sort(reverse=True)
3
4     total = 0
5
6     for weight in weights:
7         if total + weight <= capacity:
8             total += weight
9
10    return total
11
12
13 # Sample Input
14 weights = [10, 20, 30, 40, 50]
15 capacity = 60
16
17 print("Maximum weight:", maximum_weight(weights, capacity))
```

Output

Maximum weight: 60

=== Code Execution Successful ===



main.py

Visualize

Run

32

def minimum_time(jobs, k):

33

jobs.sort(reverse=True)

34

35

low = max(jobs)

36

high = sum(jobs)

37

38

while low < high:

39

mid = (low + high) // 2

40

41

if can_assign(jobs, k, mid):

42

high = mid

43

else:

44

low = mid + 1

45

46

return low

47

48

49

Sample Input

50

jobs = [3, 2, 3]

51

k = 3

52

53

print("Minimum maximum working time:", minimum_time(jobs, k))

Output

Minimum maximum working time: 3

=== Code Execution Successful ===



main.py

Visualize

Run



JS

TS

```
1 def kruskal(n, edges):
32     for u, v, w in edges:
33         if union(u, v):
37             if len(mst) == n - 1:
38                 break
39
40     return mst, total_weight
41
42
43 # Sample Input
44 n = 4
45 edges = [
46     (0, 1, 10),
47     (0, 2, 6),
48     (0, 3, 5),
49     (1, 3, 15),
50     (2, 3, 4)
51 ]
52
53 mst, weight = kruskal(n, edges)
54
55 print("Edges in MST:", mst)
56 print("Total weight of MST:", weight)
```

Output

Edges in MST: [(2, 3, 4), (0, 3, 5), (0, 1, 10)]
Total weight of MST: 19

=== Code Execution Successful ===



main.py

Visualize

Run

```
72 # Sample Input 1
73 n = 4
74 edges = [
75     (0, 1, 10),
76     (0, 2, 6),
77     (0, 3, 5),
78     (1, 3, 15),
79     (2, 3, 4)
80 ]
81
82 given_mst = [
83     (2, 3, 4),
84     (0, 3, 5),
85     (0, 1, 10)
86 ]
87
88 unique, alternative = check_unique_mst(n, edges, given_mst)
89
90 print("Is the given MST unique?", unique)
91
92 if not unique:
93     print("Another possible MST:", alternative)
```

Output

Is the given MST unique? True

=== Code Execution Successful ===